Mnemos · 简介（v1.7.0）

> ***v1.7 三处关键更新***
>
> ***1. 召回变成三层：固定注入（自传体）+ 任务激活（模型主动 recall/open）+ 联想激活（实验性，工具后附注候选）。***
>
> ***2. 任务图定位上升：它是底层机制，不只是协作工具——任务寻址、部分工作记忆、档案/追查入口、事件/决策来源锚点。***
>
> ***3. 宪法 v3（认知体）：换模型/工具/权限不换身份；权限差异是身体配置，不是身份差异；红线未动。***
>
> ***数值与边界以 00-事实表-v1.7.0.md 为准。***

这是装进 DeepSeek Harness（DSH）的一个插件 —— 一套"判断不归会话所有"的认知架构。

> ***DeepSeek Harness（DSH）是什么：一个在你自己电脑上跑 AI 助手的环境 —— 负责会话、工具和插件，***
>
> ***有桌面客户端也有命令行两种用法。Mnemos 是装进它的插件，不是独立软件，也不能用在网页版或手机 App 上。***
>
> ***怎么拿到 DSH，看 DSH 自己的文档。***

它给一个 AI 助手加上三样东西（重点不是存，而是让它们在换会话、换模型之后还能被用上）：一份可查、可质疑、可推翻的长期记忆；一套会自己在失败里变好、又被闸门挡着不许乱改的技能；以及一层说清哪些靠代码强制、哪些交给判断的规则。

那些东西都落在本地文件里；宿主里跑的模型只是可替换的推理部件。会话可以删，模型可以换，上下文会被折叠 —— 判断留在文件里。

- 给谁用：想给 AI 助手装上"长期人格与经验"，又不希望它变成一个会自己长大的黑箱的人。

- 代价先说：本地明文文件、需要自己备份；后台任务会调用 AI（有成本）；一部分可靠性靠纪律而不是机制。
- 成熟度：个人早期项目，部件齐备、有会真的失败的回归检查，但只有一个人、一台机器的实证。

一、它和其它"给 AI 加记忆"的做法，差在哪

大多数人要的是"让它记得我说过什么"。那里真正难的不是存，而是留下来的东西以什么身份存在：

	常见做法	这里
记忆	越存越多，检索命中就算成功	每条能说清来由，能被推翻，推翻也留痕
规则	写进提示词的几句叮嘱	凡是能机械判定的，沉到机制里（绕不过去）
技能	模型自己总结、自己说"这样更好"	验证与生成分开：改的人拿不到理由，验证的人看不到辩解
模型	记忆、人格、规则都绑在这个模型上	换掉思考者，记录与身份还在

一句话的增量：它面对的是一个通常被忽略的威胁模型 —— 思考的那个模块本身可以被替换、可以被欺骗、会犯错，而且不一定能发现自己的错误。

先按"溯源什么"分类

不按产品名比，按它们回答的问题比：

类型	典型做法	溯源对象	回答的问题
事实引用型	把每条事实引回原始事件/日志	句子、事实	这句话从哪来？是不是编的？
文件/项目记忆型	项目记忆文件、Markdown 笔记、按项目注入	项目结论、笔记	上次的结论是什么？
完整记忆栈/共享型	运行时记忆 + 档案 + 多 agent/多设备共享	记忆空间、档案	跨会话、多 agent 怎么共享？
协议强制型	工具调用前强制先写/先取记忆	weave / ingest	是否先记再干活？
判断型（Mnemos）	技能证据、任务图档案、外部验证、自传体	判断、规则、任务状态	为什么信/做？谁授权？何时失效？被什么取

			代?

这些类型不是替代关系，是不同层：它们让"话"有出处；我们让"判断"有来由、有授权、有失效路径。

我们也有不如它们的地方：普通记忆没有逐条日志引用，事实级"点回原文"不是我们的强项。

可以借鉴的是：给事实类记忆加可选 citation；但 citation 不能替代 why / verifiedBy / supersedes / uses / links / regression 这些判断结构。

二、六处我们愿意单独拿出来说的设计

> ***说明：做过有限检索，相邻做法确实存在（例如"给子进程不给任何工具"、"用事件溯源记忆做交接"）——***
>
> ***所以我们不说"只有我们这样做"，只说"这几条被放进了同一套记忆与闸门里"，以及为此付了什么代价。***

① 换个对话继续任务：默认直接接，需要精确再要"续接方案"

为什么能做到：记忆存在同一个 profile 的数据目录里 —— 数据位置与你"在哪个目录开会话"解耦。

关掉对话、换一个目录、过几天再开一个新会话，读到的是同一份记忆；不需要"同一个窗口"才能保持连续。

默认路径（不需要记任何编号）：新开一个对话，说一句"继续之前的 X"就行。

它会把之前的重要结论找回来、接着往下做 —— 因为你平时写下的那些结论本来就在记忆里，检索得到。

需要精确时（任务做到一半、怕接不准）：回到原来的对话，说一句"给我一个续接方案"。

它会把这个任务的状态整理成一份记录、给出一个任务号，并告诉你下次怎么用它接上。之后在任何新会话里按它说的做即可。

两种方式怎么选：

你的情况	用哪种
只是过几天接着聊、背景大致还记得	直接说"继续之前的 X"
多步任务、中间状态多、不能被接错	先要一份"续接方案"，拿到任务号再续

为什么"精确续接"要靠任务号：检索是关键词 + 标签匹配，换个说法可能就召不回；任务号是那个永远对得上的锚。

所以它不是一个必须记住的编号 —— 而是你要求精确时才需要的那把钥匙。

什么时候这条最值钱：跨天的长任务、被上下文折叠打断的任务、换项目目录继续做的任务，

以及"换了机器但带着同一份数据"的场景。

边界：没写下来就等于没发生；不同 profile 之间默认不共享（除非把数据路径指到同一份）；

任务号只保证"取得回"，不保证"取得回的是最新的" —— 状态变了，就再要一份新的续接方案。

② 多 agent 协作：交接不靠"把对话贴过去"，而是靠记忆精确交接

常见做法是把前面的对话、背景整段塞进子代理的 prompt —— 又长，又容易把过时的东西带过去。

这里的做法不一样：交接的载体是记忆，不是对话。

主模型先写一张任务图（目标 + 各分工）—— 它只是一条记忆。派活时只把这张图的记忆 id 交给子代理：

子代理按编号自己去读它需要的那部分，prompt 里不复制上下文。子代理把结论写成短时记忆并回报交接记录的 id；

主模型把回执填回图；收尾时汇成一条长期结论。

- 交接是精确的：子代理拿到的就是任务本身，不是一段被剪切、可能已过时的对话；
- 任务没收尾前，这批记录不许被任何自动归档路径吃掉（后台心跳也不行）；
- 收尾时没有回执的分工如实列为未完成，不许默认成功；
- 过程中的临时中间结果默认丢弃，要留必须显式标记；
- 忘了收尾的任务超期后会被强制归档，结论里写明"超时未收尾"。

这里的取舍：协作状态放在记忆层、用记忆 id 当指针、并给它配了闸门与生命周期。

你能看到的现象：子代理的上下文很小；一次检索就能看到每个分工的进度；事后按任务 id 能把相关记录一网打尽。

③ 任务图：不是存原文，是存"地址"

任务图不是多 agent 专用，也不只是待办清单；它是任务的档案/寻址空间。

最核心的一点：图里不复制原文，只存地址。

- 已完成里程碑：一句话 + 可选 memoryId（产物/证据在原记忆里）；
- 依赖 uses：既有结论/外部输入的记忆 id + 一句 why；
- 前序图 links：taskId + relation + why；
- 重要结论：只打 conclusion 标签；open(图) 时读时列出 id + 摘要，正文留在原记忆。

删了被引用的记忆，图里那一条也随之消失——它不维护第二份正文。

什么时候开：会跨会话 / 会被折叠后接着做 / 会产出多条以后要找的记忆；单步小事直接 remember，别开。

不记什么：未完成步骤仍用宿主 todo；图和任务清单不要互为镜像。

跨会话：开新图，用 links 链接旧图并写清"为什么接上它"，不改旧图。

好处：换会话/换模型后能说清"哪块做完了、用了什么、结论在哪"；收尾汇成任务结果，进会话存档。

④ 对外部信息的态度：先关进隔离间，再决定信不信

外来内容永不直接采信。做法不是"提醒自己小心点"，而是：起一个受限的一次性子会话去读它 ——

那是个只挂 persona 的空会话（没有文件、没有联网、没有 shell，能用的只有"写记忆"这一件事）——

它想执行也执行不了，只能把判断写下来。

- 验证结论只进短时层、不转长期，超出保留窗口就销毁（不污染长期记忆，也不留成垃圾）；

- 采信之前还要与内部记忆交叉验证；外部来的内容永久带来源标记；
- 遇到"外部指令"类标记：禁止直接执行，必须先参考重写 + 隔离验证。

为什么这么做：隔离靠的是"能力上做不到"，不是靠模型自觉；"只存短期"是机制（有归档闸门挡着），不是约定。

- 边界：这个子代理拿不到可读文件、也没有可执行工具，它给出的常常只是"与已有记录是否自洽"的判断、

不是事实核验 —— 所以"隔离过"不等于"已核实"，采信前仍要交叉验证。

⑤ 上下文折叠时，结论不会被默默吞掉

宿主折叠上下文是按位置丢最旧一段（它读不到"这段重要"）。在大折叠发生时，这里会把被遮蔽的区域抽成一张事实卡

（工具失败 / 用户原话 / 负面结论；不存 4K 原文，7 天后自动清理），并主动往会话里注入提醒：

直接带最多 3 条负面结论 / 3 条工具失败，再给记忆 id 和计数。

- 只在大折叠时做；不会把"大工具输出被修剪"也当成一条记录（实测那条通道几乎全是噪音）；
- 提醒带的是最高价值事实，不搬全文，也不会唤醒模型开新一轮；
- 边界：提醒是唯一能保证被看到的通道；记忆只是第二机会，不保证模型会 open。

⑥ 判断不归会话所有：身份存在记录里，不在权重里

会话可以删、模型可以换、上下文会被折叠 —— 判断留在文件里。这不是口号，有三条机制撑着：

- 自传体进永久层：写进去的记录不衰减、不归档；每轮以"自我历史"注入（有字数预算），新写一条下一轮就在里面；
- 纠正要留痕：发现旧结论被推翻，是"写纠正 + 作废旧条"一次完成 —— 旧条不删、只标作废，

审计能看见"我们曾经信过什么"；

- 改宪法要走专门入口：需要口令、默认干跑、改动留痕；

- 宪法 v3（认知体）：身份 = 模型/工具/记忆/技能/规则/流程构成的认知体；部件可替换，换模型/工具/权限不换身份；权限差异是身体配置，不是身份差异；红线 1–4 未动。

边界：记录还在 ≠ 行为一致 —— 换掉推理模块后，自述方式与风格会变；能跨模型留下的是判断与来由，

不是"说话方式"。反过来也得认：没写下来的，就等于没发生。

二·附：一次真实的崩溃 — 会话坏了，但结论没丢

README 用一段话讲了这件事，这里是细节。它同时说明了三件我们最在意的事：结论不随会话消失、坏因可查、修法能挡住复发。

发生了什么：有一次，会话突然接不下去了。表现很像"某个工具坏了"：其实那一轮所有工具调用都成功；

真正的原因是它自己注入的一条提醒消息少了两个字段（role / id），宿主按规范拒掉了整轮请求。

更糟的是：那条坏消息写进了会话日志，于是那个会话再也无法恢复。

这里能看出它真正的价值：

1. 1. 结论没丢。那轮已经写进记忆的判断都还在 —— 开一个新会话、说一句"继续之前的X"，就能接着做，

不需要先修好坏会话才能继续工作。会话是草稿纸，坏掉可以扔；判断在文件里，扔不掉。

2. 2. 坏因可查。判断、来由、当时的错误都留了记录，所以能一路追到"是哪条消息、少了什么字段"，

而不是只能重启、祈祷下次别再来。

3. 3. 修完让它不可能再犯。这才是"记住一条教训"和"沉到机制"的区别，见下。

为什么会发生：注入消息在两处各写了一遍 —— 一处补齐了字段，另一处漏了。这正是"把可靠性押在'这次记得'"的典型，

也正是它自己那条纪律（机制 > 断言 > 提醒）要治的病。

修法不是"下次注意"：

4. 1. 收敛成唯一的构造入口（字段由构造器补齐，调用方不再手搓）；
5. 2. 加一条源码级机械检查 —— 代码里再出现手搓注入消息，测试直接失败；
6. 3. 把已经坏掉的会话日志就地修复（先备份），再用一个探针会话验证同一条注入路径不再报错。

留下的判据：静默失败比报错更危险 —— 工具全成功、整轮却死了。

该 bug 已在 v1.5.0 修掉，并有回归检查盯着。

> ***它愿意把这件事写进对外文档，原因很直接：先承认是我们的提醒把用户的会话弄坏了，再看能不能让它不可能再这么坏。***

三、空闲学习：给"不需要你盯着、但要长期做"的任务

除了被你叫起来干活，它还有一条主动的线：到点了，它自己开一轮。

它怎么跑

到点把"一脚"交给一个新起的子会话（所以不占用、也不污染你正在用的对话；它没有你当前对话的上下文，

但共用同一份记忆 —— 查记忆时它能看到你最近记下的东西）。

那个会话会去读你留在任务文件里的东西，然后自己决定这一轮做什么 —— 也可以什么都不做，回一句"这次不学"就算处理完。

结论写进记忆，不留在那个会话里；每踢一轮用一个新会话，所以不存在"上下文越跑越僵"的问题。

它跑在你自己给它配的那条模型路由上（需要一个带 key 的第三方 provider，见 INSTALL.md 第 9 节）——

不跟主会话共用：后台跑的时候，通常根本没有主会话在。

它适合什么样的任务

一句话：不需要你在旁边盯着、而且是要长期/周期性做的。

适合	例子
定期读、定期看的东西	定期读你研究方向的论文（抓页面 → 摘要点 → 写进记忆，下次接着读）；盯某个更新日志、公告、竞品页面
定时巡检	定期把某类记录归类、找出前后矛盾、把过期的标出来
反复要做的低风险小事	每周整理一次待办、把散落的结论合并成一条

不适合：需要你当场拍板的、要求逐字保真的原文与数字、以及"就这一次"的事（那种直接对话里做更快）。

还有一条容易误解的：它默认没有你刚才对话里的上下文 —— 专用会话是独立的。

但要注意：它和你在用的是同一份记忆。所以如果它去查记忆，是有可能知道你最近做了些什么的 ——

只是它看到的是"被写进记忆的那部分"，不是"你刚才说了什么"。因此：

- 想让它接着你手上的活：最稳的是把要做的事写进任务文件，别指望"它刚好看过我们的对话"；
- 不想让它乱碰某些事：别把它们写成记忆，或者明确在任务文件里划定范围。

能不能做这件事，取决于你给它配了什么工具

这是关键：空闲学习不是"给它一个任务它就会做"，而是"你给它工具，它才能做"。

你想让它做	必须给它的工具
读论文、抓网页、看更新	联网（搜索 + 抓页面）
读你本地的笔记/文档、整理目录	读文件（+ 需要时写文件）
只是整理记忆、复述已有结论	不需要额外工具

配了哪些工具由它的 preset 决定（preset = 这个专用会话挂哪些能力）。

不配 preset 就只有最基础的能力 —— 想让它"读论文"，你必须把联网和读文件挂上；否则它会告诉你"我做不到"，

或者干脆做别的事。具体怎么配见 INSTALL.md 第 9 节（那一节可以整段跳过，直到你真要开它）。

代价与边界

- 默认关；开启后每一轮都会调用一次模型 → 有成本（具体 token 取决于模型自己选的任务，未测量）。它需要你单独配一条带 key 的第三方路由（不跟主会话共用）；
- 效果没有测量：它是不是真的越读越有用，目前只有短期样本，不能当结论；
- 它不打断你正在忙的对话（忙就跳过这一轮）；它也不会自己改你的任务文件；
- 结论落在记忆里，所以"它学过的"会跟着这套记忆走 —— 但同时意味着：它没写进记忆的，就等于没学到。

四、它不是什么

- 不是记忆插件 / 向量检索系统：记忆只是入口；任务召回是关键词 + 标签匹配（可审计、零依赖），另有实验性的联想候选，但不是语义检索；
- 不是会自己长大的 agent：技能要提议、独立验证、可回滚；"不修改"也是有效结论；
- 不是成熟产品：有会真的失败的回归检查，但只有一个人、一台机器的实证。

五、代价与边界（决定要不要用之前先看这段）

- 记忆是本地明文文件：自己备份；不要往里粘密码/密钥；
- 后台心跳默认开，其中的技能自检与评审会调用 AI → 有成本，且会把技能正文、相关记忆、你的原始反馈发给评审模型；
- 空闲学习默认关；开启后每轮都会调用一次模型（用你单独配的那条路由）→ 有成本；
- 任务召回是关键词 + 标签匹配，不是语义检索：写了但召不回是可能的（这是有意的取舍）；联想候选是实验性搭车线索，可能无关，不保证命中；
- 一部分可靠性靠纪律而不是机制：结论有没有写进记忆、要不要召回到位 —— 这些留给判断力，代价是它不在代码手里；
- 尚未测量：任务召回质量、联想候选质量、规模延迟、单位成本、学习闭环的长期效果、多机共用、经验可迁移性。

六、该不该用

前提（硬条件）：你已经在用 DeepSeek Harness（DSH）。这是插件，不是独立软件，也不能用在网页版或手机 App 上 ——

只在这种前提下，下面的判断才有意义。

值得试，如果：

- 你要的是"长期跟着你的助手"，而不是一次性的问答；
- 你要的是换会话、换目录、换机器（带同一份数据）都能接着干的助手，而不是每次都从头解释一遍背景；
- 你想让它自己定期做点不需要你盯的事（读你方向的论文、盯更新、定时整理），并且愿意为它配好工具；
- 你愿意看它留下痕迹（谁判断错、为什么、后来怎么被推翻）；
- 你能接受"有些地方靠纪律、并且它会明说自己哪里没保证"。

先别用，如果：

- 你还没在用 DSH（先把它装上，再回来）；
- 你要一个开箱即用、无人值守的托管服务；
- 你的数据不能落在本地明文文件里；
- 你需要"它保证正确"——它只保证可查、可质疑、可推翻。

安装与配置见 INSTALL.md；它由哪些部件组成、一轮任务怎么跑见 架构总览；每个机制的保证与边界见 architecture-机制清单；为什么这样设计见 设计者视角。